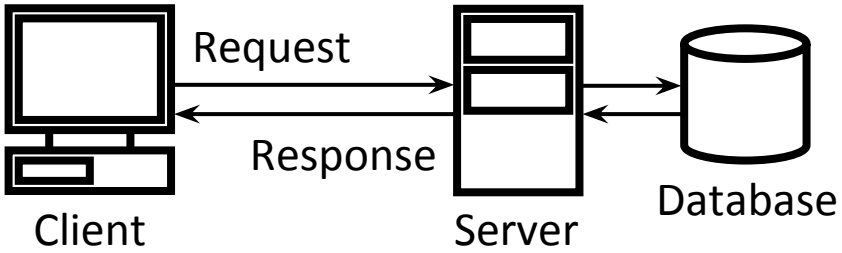
Chapter 6: Web-Based Database Programming

- PHP Data Object (PDO)
- SELECT, INSERT, UPDATE and DELETE
- Prepared Statement and Database Transaction
- Database Security Practices

References:

- https://phpdelusions.net/pdo
- https://www.php.net/manual/en/book.pdo.php

1. PHP and Database



Database Examples:
- MySQL / MariaDB
- SQLite
- Oracle, etc.

- Web-based systems persist data permanently in database.
- Commonly used databases for PHP:
  - **MySQL** / **MariaDB** = A full-featured database server.
  - **SQLite** = A light-weight standalone embedded database (not a database server). OK for low to medium traffic websites.

NOTE:
- **MySQL** = Developed by MySQL AB in 1995, which acquired by Sun Microsystems in 2008, which in turn acquired by Oracle Corporation in 2010.
- **MariaDB** = Community-developed fork of MySQL, led by original developers of MySQL, to keep the project free and open-source. XAMPP uses MariaDB.

1. PHP and Database…

- One can use the following libraries / extensions to connect and operate with database in PHP:
  - MySQL Original = **`mysql`**
    - Deprecated in PHP 5.5. Removed in PHP 7.0.
  - MySQL Improved = **`mysqli`**
    - Can work with MySQL database only.
  - ✅ PHP Data Object = **PDO** ← We learn this…

Advantages

- Use object-oriented approach in coding.
- Can work with different databases (e.g. MySQL, SQLite, Oracle, etc.) if database-specific PDO drivers are available.
- Provide data-access abstraction layer. Regardless which database is used, we use the same **PDO** functions to issue queries and fetch data.
- Changing database (e.g. from MySQL to SQLite) usually only requires minimum rewrite to the database-specific SQL statements used to suit the targeted database.

2. Connect to Database

- To connect to a database, we supply **DSN (data source name)** to the **PDO** constructor, together with username, password and <u>optional</u> options:

Database driver prefix

DSN (data source name)

Username

Password

```
$_db = new PDO('mysql:dbname=db;host=localhost;port=3306', 'root', '', [
  PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_OBJ,
]);
```

Optional connection options as KEY => VALUE array

- <u>NOTE:</u>
  - MySQL runs on port **3306** by default.
  - Default username for MySQL is **'root'**.
  - Default password for MySQL is **'' (empty string)**.
  - The **`PDO::FETCH_OBJ`** option instructs **PDO** to return records as object (rather than the default associative array).

2. Connect to Database…

- **DSN (data source name)**:
  - The database server may locate on same or different machine.
  - If the database is at **localhost:3306**, the codes can be simplified:

```
$_db = new PDO('mysql:dbname=db', 'root', '', [
  PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_OBJ,
]);
```

- <u>NOTE:</u>
  - Different databases have different DSN format.
  - Reference: https://www.php.net/manual/en/pdo.drivers.php
  - For example, to connect to SQLite database:

```
$_db = new PDO('sqlite:database.db', '', '', [
  PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_OBJ,
]);
```

3. SELECT Record

- To SELECT and return <u>multiple</u> records:

db **fruit**
- id : int(11)
- name : varchar(100)
- price : decimal(4,2)

```
$stm = $_db->query('SELECT * FROM fruit');
$arr = $stm->fetchAll();

foreach ($arr as $f) {
  echo "$f->id | $f->name | $f->price<br>";
}
// 1 | Apple | 1.00
// 2 | Banana | 2.00
// 3 | Coconut | 3.00
```

- The **`query()`** function returns a **`PDOStatement`** object.
- Then call **`fetchAll()`** to return an array of records.
- The functions above can be chained into a single statement:

```
$arr = $_db->query('SELECT * FROM fruit')
           ->fetchAll();
```

3. SELECT Record…

- To SELECT and return <u>one</u> or <u>first</u> record:

db **fruit**
id : int(11)
name : varchar(100)
price : decimal(4,2)

```
$f = $_db->query('SELECT * FROM fruit')
         ->fetch();

echo "$f->id | $f->name | $f->price<br>";
// 1 | Apple | 1.00
```

- To SELECT and return <u>one</u> or <u>first</u> **column**:

```
$id = $_db->query('SELECT * FROM fruit')
          ->fetchColumn();

echo "$id<br>";
// 1
```

- This is useful if for query that returns single value. For example:

```
$count = $_db->query('SELECT COUNT(*) FROM fruit')
             ->fetchColumn();
```

3. SELECT Record…

- To SELECT and return **key-value pairs**:
  - Sometime, it is useful to return records as key-value pairs in an associative array (e.g. to populate a selection list with items).

NOTE: The query must return exactly 2 columns

```
$items = $_db->query('SELECT id, name FROM fruit')
             ->fetchAll(PDO::FETCH_KEY_PAIR);
...
<?= html_select('fruit', $items) ?>
```

Key	Value
1	Apple
2	Banana
3	Coconut

NOTE:

- Some other PDO fetch modes:
  - Fetch mode can be applied to both of the **`fetch()`** and **`fetchAll()`** functions.

Fetch Mode	Description
`PDO::FETCH_DEFAULT`	Use the default fetch mode as in PDO options.
`PDO::FETCH_ASSOC`	Return each record as an associative array.
`PDO::FETCH_NUM`	Return each record as an indexed array.
`PDO::FETCH_BOTH`	Equivalent to `FETCH_ASSOC` + `FETCH_NUM`.
`PDO::FETCH_OBJ`	Return each record as an object.
`PDO::FETCH_COLUMN`	Return only a single column.
`PDO::FETCH_KEY_PAIR`	Return each record as a key-value pair.
`PDO::FETCH_UNIQUE`	Return records indexed by the first column in the query. If the value is duplicated, later will overwrite.
`PDO::FETCH_GROUP`	Group records by the first column in the query. Each group is maintained as a nested array.

3. SELECT Record…

- Summary:
  - **`fetchAll()`** = Return multiple records as an array of objects.
  - **`fetch()`** = Return one record as an object.
  - **`fetchColumn()`** = Return one scalar value.

- References (fetch modes):
  - https://phpdelusions.net/pdo/fetch_modes
  - https://www.php.net/manual/en/pdo.constants.php

4. Prepared Statement

- Use **prepared statement** for **parameterized query**.
  - Parameterized query = query that contains parameters (or placeholders) for input values. For example:

- **[1] Positional parameters:**

Positional parameter

```
$id = 1;
$stm = $_db->prepare('SELECT * FROM fruit WHERE id = ?');
$stm->execute([$id]);
$f = $stm->fetch();
// 1 | Apple | 1.00
```

Values are supplied by the position of the parameters in order (as indexed array)

- **[2] Named parameters:**

Named parameter

```
$id = 1;
$stm = $_db->prepare('SELECT * FROM fruit WHERE id = :id');
$stm->execute(['id' => $id]);
$f = $stm->fetch();
// 1 | Apple | 1.00
```

Values are supplied by the name of the parameters (as associative array)

4. Prepared Statement...

- Example:
  - **[1] Positional parameters:**

We will use positional parameter as it is more concise

```
$min = 1.00;
$max = 3.00;
$stm = $_db->prepare('SELECT * FROM fruit
                      WHERE price >= ? AND price <= ?');
$stm->execute([$min, $max]);
$arr = $stm->fetchAll();
```

NOTE: The **`execute()`** function returns **`true`** for successful execution or **`false`** otherwise

  - **[2] Named parameters:**

```
$min = 1.00;
$max = 3.00;
$stm = $_db->prepare('SELECT * FROM fruit
                      WHERE price >= :min AND price <= :max');
$stm->execute(['min' => $min, 'max' => $max]);
$arr = $stm->fetchAll();
```

4. Prepared Statement…

- NOTE:
  - The **`prepare()`** function prepares a parameterized query BUT DOES NOT execute it. It returns a **`PDOStatement`** object.
  - Call the **`execute()`** function to execute the query by supplying parameters as an array.
  - If the prepared statement has NO parameter:

```
$stm->execute();    // Leave it null
$stm->execute([]); // Or empty array
```

  - Generally, **`query()`** = **`prepare()`** + **`execute()`** but without parameters (or placeholders).

```
$stm = $_db->query('SELECT * FROM fruit');
$arr = $stm->fetchAll();
```

Same as…

```
$stm = $_db->prepare('SELECT * FROM fruit');
$stm->execute([]);
$arr = $stm->fetchAll();
```

4. Prepared Statement...

- NOTE:
  - A prepared statement can be executed multiple times with different parameter values. For example:

```
$data = [
  'Watermelon' => 4.00,
  'Strawberry' => 5.00,
  'Tangerine'  => 6.00,
];

$stm = $_db->prepare('INSERT INTO fruit (name, price)
                      VALUES (?, ?)');

foreach ($data as $name => $price) {
  $stm->execute([$name, $price]);
}
```

id is an **AUTO INCREMENT** field

db **fruit**
- id : int(11)
- name : varchar(100)
- price : decimal(4,2)

Execute multiple times with different parameter values

4. Prepared Statement…

- NOTE:
  - SQL parameters only work for data values (i.e. string and numeric literals).
  - You CANNOT use SQL parameters to substitute SQL keywords, functions, table names and field names.

```
$stm = $_db->prepare('SELECT * FROM fruit WHERE id = ?');
$stm->execute([1]); // OK
$arr = $stm->fetchAll();
```

Table name
⍰ NOT allowed

Field name
⍰ NOT allowed

```
$stm = $_db->prepare('SELECT * FROM ? WHERE ? = ?');
$stm->execute(['fruit', 'id', 1]); // FAILED
$arr = $stm->fetchAll();
```

4. Prepared Statement…

- NOTE:

NOTE: Use with EXTRA CAUTION

  - Use **PHP variable substitution technique** if you REALLY need to modify non-value parts of the query. For example: if you need to sort the records by different fields and orders.

NOTE: MUST perform **WHITELIST CHECKING** if the values are coming from untrusted sources (e.g. user inputs, URL, etc.)

WHITELIST CHECKING = Only acceptable values are allowed to pass through. REJECT all others

```
$field = 'price';
$order = 'DESC';

in_array($field, ['id', 'name', 'price']) || $field = 'id';
in_array($order, ['ASC', 'DESC']) || $order = 'ASC';

$arr = $_db->query("SELECT * FROM fruit
                    ORDER BY $field $order")
           ->fetchAll();
```

PHP variable substitution

SECURITY CONCERN

- Prepare statements will automatically escape any special characters in the SQL parameters. Hence help to prevent common database security flaw such as **SQL injection**.

Advantages

- DO NOT use **PHP variable substitution technique** for SQL statements whenever possible. If you REALLY need to do so, perform thorough WHITELIST CHECKING to the values and prevent unwanted values from passing through to the SQL statements.

SECURITY CONCERN…

- Example (SQL Injection):
  - Assume the following codes:

UNCHECKED PHP variable substitution in SQL statement

```
$id = req('id'); // Data from untrusted source
$f = $_db->query("SELECT * FROM fruit WHERE id = $id")->fetch();
```

  - In ideal situation:

```
http://localhost:8000/index.php?id=1
```

```
SELECT * FROM fruit WHERE id = 1
```

  - BUT a hacker will submit the followings:

```
http://localhost:8000/index.php?id=1; DELETE FROM fruit
```

```
SELECT * FROM fruit WHERE id = 1; DELETE FROM fruit
```

  - It ends up all fruit records are deleted (and it can be worse).
  - **Prepare statements** can prevent this issue automatically.

SECURITY CONCERN…

- You may use the **PDO** object's **`quote()`** function to place quotes around the input string (if required) and escape special characters within the input string.
- The **`quote()`** function returns a quoted string that is theoretically safe to pass into an SQL statement.
- For example:

NOTE: NO NEED to do this if you are using **prepared statement**

Data type constants:

- `PDO::PARAM_BOOL`
- `PDO::PARAM_INT`
- `PDO::PARAM_LOB`
- `PDO::PARAM_NULL`
- `PDO::PARAM_STR`

```
$id = req('id'); // Data from untrusted source
$id = $_db->quote($id, PDO::PARAM_INT);
$f = $_db->query("SELECT * FROM fruit WHERE id = $id")->fetch();
```

5. The bindValue() Function

- One can also call **`bindValue()`** function to bind a value to a parameter one-by-one. For positional parameters:

```
$min = 1.00;
$max = 3.00;
$stm = $_db->prepare('SELECT * FROM fruit
                      WHERE price >= ? AND price <= ?');
$stm->bindValue(1, $min, PDO::PARAM_STR);
$stm->bindValue(2, $max, PDO::PARAM_STR);
$stm->execute();
$arr = $stm->fetchAll();
```

Parameter position (start from 1)

Value or variable

Parameter data type

Data type constants:

- PDO::PARAM_BOOL
- PDO::PARAM_INT
- PDO::PARAM_LOB
- PDO::PARAM_NULL
- PDO::PARAM_STR

- Use **PDO::PARAM_LOB** for **TEXT** and **BLOB** types.
  - **TEXT** for large chunk of text up to 4GB (**VARCHAR** max 64KB).
  - **BLOB** (binary large object) for binary data.
- Use **PDO::PARAM_STR** for **DECIMAL**, **FLOAT** and **DATETIME**.

5. The bindValue() Function…

- For named parameters:

```
...
$stm = $_db->prepare('SELECT * FROM fruit
                      WHERE price >= :min AND price <= :max');
$stm->bindValue('min', $min, PDO::PARAM_STR);
$stm->bindValue('max', $max, PDO::PARAM_STR);
...
```

- <u>NOTE:</u>
  - We will stay with supplying parameters via the **`execute()`** function as it is more concise.
  - There is also the **`bindParam()`** function, which allows 2-way value binding (i.e. input and output parameter). We will skip this however.

6. INSERT record

- To INSERT a record:

```
$id = 4;
$name = 'Durian';
$price = 4.00;

$stm = $_db->prepare('INSERT INTO
                      fruit (id, name, price)
                      VALUES (?, ?, ?)');
$stm->execute([$id, $name, $price]);
```

id is an **AUTO INCREMENT** field

db **fruit**
id : int(11)
name : varchar(100)
price : decimal(4,2)

- <u>NOTE:</u>
  - You can optionally omit the AUTO INCREMENT field.
  - If this is the case, the database will auto generate the id.

6. INSERT record...

- To INSERT a record with auto id:

```
$name = 'Elderberry';
$price = 5.00;

$stm = $_db->prepare('INSERT INTO
                      fruit (name, price)
                      VALUES (?, ?)');
$stm->execute([$name, $price]);

$id = $_db->lastInsertId();
echo $id; // 5
```

id is an **AUTO INCREMENT** field

db **fruit**

id : int(11)

name : varchar(100)

price : decimal(4,2)

- <u>NOTE:</u>
  - Call the **PDO** object's **`lastInsertId()`** function to obtain the id of the last inserted record. Useful for getting the auto id.

7. UPDATE record

- To UPDATE a record:

```
$id = 4;
$name = 'Durian (Musang King)';
$price = 90.00;

$stm = $_db->prepare('UPDATE fruit
                      SET name = ?, price = ?
                      WHERE id = ?');
$stm->execute([$name, $price, $id]);
```

db **fruit**

- id : int(11)
- name : varchar(100)
- price : decimal(4,2)

7. UPDATE record…

- To UPDATE multiple record:
  - For example: To increase all prices by 1.00…

```
$stm = $_db->prepare('UPDATE fruit
                      SET price = price + 1.00');
$stm->execute();

$count = $stm->rowCount();
echo "$count record(s) updated"; // 5 record(s) updated
```

  - The **`PDOStatement`** object's **`rowCount()`** function returns the number of rows affected by the last SQL statement.
  - As there is no SQL parameter, you can simply use the **`query()`** function to do the same without the **`execute()`** function:

```
$stm = $_db->query('UPDATE fruit
                    SET price = price + 1.00');

$count = $stm->rowCount();
echo "$count record(s) updated"; // 5 record(s) updated
```

8. DELETE record

- To DELETE a record:

```
$id = 4;

$stm = $_db->prepare('DELETE FROM fruit WHERE id = ?');
$stm->execute([$id]);
```

NOTE: DELETE statement usually comes with a WHERE clause (or ALL records will be deleted)

- To DELETE all records from a table:

```
$stm = $_db->query('DELETE FROM fruit');

$count = $stm->rowCount();
echo "$count record(s) deleted"; // 5 record(s) deleted
```

NOTE:

- The **PDO** object's **`exec()`** function execute a query and return the number of rows affected by the query.
- It is a shortcut for **`query()`** + **`rowCount()`**. Useful if:
  - There is no SQL parameter.
  - There is no record returned (i.e. not a SELECT statement).
  - You want to get the number of rows affected.

```
$stm = $_db->query('DELETE FROM fruit');
$count = $stm->rowCount();
echo "$count record(s) deleted"; // 5 record(s) deleted
```

Same result...

```
$count = $_db->exec('DELETE FROM fruit');
echo "$count record(s) deleted"; // 5 record(s) deleted
```

9. Database Transaction

- We use a **transaction** to group a sequence of related database operations together as a single unit.
- A transaction ensures that:
  - Either all database operations are completed successfully and all changes are committed to the database as a whole.
  - Or if any of the operations failed, then no changes shall be made to the database at all.

- We can use the following **PDO** object's functions to work with transaction:
  - **`beingTransaction()`** = Start a transaction.
  - **`commit()`** = Commit all changes made since transaction starts.
  - **`rollBack()`** = Cancel all changes made since transaction starts.

Depending on the logic, the **`rollBack()`** function may not need to be called

9. Database Transaction...

- Example:

```
$data = [
  [1, 'Apple'  , 1.00],
  [2, 'Banana' , 2.00],
  [3, 'Coconut', 3.00],
];

$stm = $_db->prepare('INSERT INTO
                      fruit (id, name, price)
                      VALUES (?, ?, ?)');

$_db->beginTransaction();
foreach ($data as $row) {
  $stm->execute($row);
}
$_db->commit();
```

db **fruit**

- id : int(11)
- name : varchar(100)
- price : decimal(4,2)

Begin transaction

Execute the SQL. BUT no record will be actually inserted until **`commit()`** is called

Commit all changes to database at once

NOTE: Transaction ensures that either all records are inserted or none at all

10. Database Security Practices

- Some practices to improve database security:

1. Use prepared statements and parameterized queries (rather than mixing SQL with variables).

2. Utilize input validations and output encodings to address special characters (e.g. use the **`quote()`** function).

3. Use stored procedures to abstract data access and disallow direct access to base tables.

4. Application should use the lowest possible level of privilege when accessing database (e.g. cannot CREATE or DROP table).

5. Create different database accounts for different user roles. Grant only the permissions needed.

10. Database Security Practices…

- Continued:

6. Change the default database administrative username and password (e.g. do not use **'root'** and **''** in production).

7. Change the default port number of the database server. If possible, host the database server on a different computer.

8. Remove unnecessary default vendor content (e.g. sample schemas or sample tables).

9. Turn off unnecessary database functionality (e.g. unnecessary services and packages).

10. Turn off or disable database client (e.g. PHPMyAdmin) when it is not in use.

10. Database Security Practices…

- Continued:

11. Utilize strong passwords. Use multi-factor authentication if the database server supports.

12. Do not hardcode database settings in codes. Store them in a separated and encrypted file.